HACKATHON • 17 OCT 2026

LINUX DEBUGGING SYSTEM

Memory Leak Detection & Reporting Tool

A C-based Linux tool to track dynamic memory, identify unreleased tracked blocks, and generate a clear report for debugging.

KL University | CSE

Guided by Dr. K. Anusha

Presented by:
2520030034 - P.PRATHIK
2520030188- U.PRANEETH
2520030573- M. HARSHITH



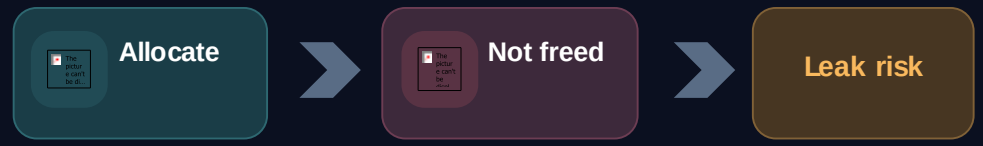
Why Memory Leaks Matter

A small memory-management mistake can quietly consume resources as a program keeps running.

THE PROBLEM

Allocated memory needs an owner.

If the program loses track of a block or forgets to release it, that memory can remain reserved.



WHAT CAN GO WRONG?

Higher memory consumption

Unused blocks accumulate over time.

Performance pressure

Less available memory can affect stability.

Harder debugging

The source of a leak may be difficult to locate.

Our goal

Track allocations, find unreleased tracked blocks, and report them clearly.

Our Goal: Make Memory State Visible

A lightweight tracking layer turns memory operations into a report developers can inspect.

Project objectives

- Record tracked allocations
- Track release operations
- Identify remaining blocks
- Calculate block count and bytes
- Save a readable report
- Integrate with Linux Debugging System

PROPOSED WORKFLOW

01 Track allocation
malloc / calloc wrappers

02 Track release
Record calls through tracker

03 Compare records
Find blocks still outstanding

04 Create report
Count blocks + bytes

Clear scope = trustworthy results

The baseline detects allocations routed through our tracking API. Arbitrary uninstrumented programs need additional instrumentation or a dedicated memory-analysis tool.

Core Features

Six practical capabilities make the tool useful during a live debugging demonstration.



01 Allocation tracking

Record pointer and size for allocations that pass through tracked malloc / calloc wrappers.



02 Release tracking

Mark tracked blocks as released when the program uses the tracked free operation.



03 Leak identification

Find registry entries that remain allocated when the report is requested.



04 Byte calculation

Summarize the number of outstanding blocks and their combined size in bytes.



05 Report generation

Print a readable console summary and save results to a text report file.



06 Linux integration

Expose the detector from the Linux Debugging System command-line workflow.

IMPLEMENTATION NOTE Only operations routed through the tracker are counted in the baseline version.

SYSTEM DESIGN

From memory operations to a report

A simple, auditable pipeline built around a tracked allocation registry.



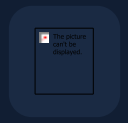
Test program

Requests memory



Tracked API

malloc / free wrappers



Block registry

Pointer + size + state



Leak checker

Find outstanding blocks



Report output

Console + text file



Linux Debugging System

Main menu / command-line entry point for the memory detector.



Safe, controlled testing

Use test programs you control. Avoid probing other users' processes or interfering with system services.

DESIGN PRINCIPLE • Observe only what the tracking layer can actually see.

Technology Stack & Implementation

A lightweight, Linux-native design built with tools that are easy to compile, inspect, and share.

C programming

Memory operations and tracking logic

Ubuntu Linux

Development and test environment

GCC compiler

Compile with warnings and debug info

Git + GitHub

Version control and collaboration

WHY THIS STACK?

Small • transparent • reproducible

The code, build steps, test cases, and output belong in the repository.

IMPLEMENTATION PATH

01	**Wrap allocation**	Record returned pointer and requested size.
02	**Wrap release**	Find the pointer and update its tracked state.
03	**Check registry**	List records that remain active at report time.
04	**Write report**	Print counts and byte totals to console / file.

Build / execution commands should be documented in README.md and verified on Ubuntu.

Testing & Validation

Use controlled test cases to confirm that the tracker flags remaining blocks and clears released ones.

TEST 01

Intentional leak

Allocate 100 bytes and do not release the tracked block.

Expected: 1 outstanding block • 100 bytes

TEST 02

Correct release

Allocate 100 bytes, then release the same tracked block.

Expected: 0 outstanding blocks • 0 bytes

TEST 03

Partial release

Allocate multiple blocks and release only some of them.

Expected: only remaining blocks listed

memory_report.txt / example format

```
MEMORY LEAK DETECTION REPORT
Total allocations       : 1
Total released blocks   : 0
Potential leaked blocks : 1
Potential leaked memory : 100 bytes
Status: MEMORY LEAK DETECTED
```

VALIDATION CHECKLIST

- Compare expected vs actual counts
- Verify byte totals match test inputs
- Confirm report file is created
- Test invalid / repeated release safely

EXAMPLE FORMAT — NOT A VERIFIED RUN

Live Demonstration Plan

A 90-second walkthrough should show the problem, the detector, and evidence that the report is correct.

01



Launch

Open the Linux Debugging System and choose the memory detector.

02



Leak case

Run a test that intentionally leaves a tracked block unreleased.

03



Inspect report

Show outstanding block count, total bytes, and saved report file.

04



Clean case

Run a second test where every tracked block is released.



WHAT THE EVALUATOR SHOULD SEE

1) leak flagged 2) correct totals 3) report file saved 4) clean test shows zero outstanding blocks

GitHub, Documentation & Teamwork

The repository should make your prototype easy to build, test, review, and run on evaluation day.



Repository checklist

```
Linux-Debugging-System/
├── src/
│   ├── main.c
│   └── leak_tracker.c / .h
├── tests/
├── reports/
├── README.md
├── PROJECT_REPORT.md
└── .gitignore  (optional Makefile)
```

README: setup • compile • run • test • sample output

TEAM CONTRIBUTION



Implementation

Memory tracker, registry, report generation



Testing & integration

Leak / no-leak cases, Linux menu integration



Documentation

README, project report, architecture diagram



Demo & presentation

PPT, live demo, clear explanation of results

Every member should be able to explain the complete flow.

IMPACT & NEXT STEPS

Turn memory operations into actionable debugging evidence

A focused first version with clear opportunities for improvement.



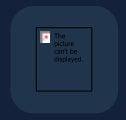
Project impact

- Makes tracked memory use visible
- Helps investigate leaks faster
- Produces a shareable report
- Supports OS / systems learning



Future enhancements

- Record source file / line locations
- Detect invalid or repeated frees
- Support more allocation APIs
- Integrate with Valgrind / ASan



Conclusion

- Track allocations and releases
- Find remaining tracked blocks
- Summarize count and byte totals
- Demonstrate with verified tests

OUR PROMISE

An explainable memory leak report — backed by controlled tests, clear scope, and reproducible code.

THANK YOU